

AI 545 MLOps

Running an Inference Service

1 Conceptual Shift: From Model Artifact to Model Service

So far:

- Git versions code.
- DVC versions data.
- MLflow versions models.
- Docker versions environments.

Now you go one level deeper.

A model file stored in MLflow is not useful by itself.

A model becomes meaningful when:

- It is loaded deterministically.
- It is wrapped behind a stable API.
- It runs in a controlled container (later)

This lecture transforms a versioned MLflow model into an inference microservice.

2 Dataset: Breast Cancer Wisconsin

You will use the Breast Cancer Wisconsin dataset from `sklearn.datasets`.

This is a real biomedical classification dataset:

- 569 samples
- 30 numeric features
- Binary classification (malignant vs benign)

You use it because:

- It is realistic.
- It requires proper feature validation.
- It simulates healthcare-like ML deployment.

3 Step 1: Train and Log Model with MLflow

Create a file:

`train.py`

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```

mlflow.set_experiment("breast_cancer_experiment")

def main():
    data = load_breast_cancer()
    X = data.data
    y = data.target
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        random_state=42)
    with mlflow.start_run() as run:
        model = RandomForestClassifier(
            n_estimators=100,
            random_state=42
        )
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        acc = accuracy_score(y_test, preds)
        print(f"Accuracy: {acc:.4f}")
        # Log test accuracy metric
        mlflow.log_metric("accuracy", acc)
        # Log and register model in the MLflow Model Registry
        mlflow.sklearn.log_model(
            model,
            artifact_path="model",
            registered_model_name="BreastCancerClassifier"
        )
        print(f"Run ID: {run.info.run_id}")

```

```

if __name__ == "__main__":
    main()

```

Now run:

```
python train.py
```

Open MLflow UI:

```
mlflow ui
```

Navigate to:

```
http://localhost:5000
```

Go to Model Registry. You should see:

BreastCancerClassifier. After the model version is created in MLflow, assign the alias champion on MLFlow

Now your best model is referenced by:

```
models:/BreastCancerClassifier@champion
```

4 Step 2: Build the FastAPI Inference Service

Create a file:

```
app.py
```

```

import os
import numpy as np
import mlflow
import mlflow.pyfunc
from fastapi import FastAPI
from pydantic import BaseModel
import warnings
warnings.filterwarnings("ignore")

app = FastAPI()

MODEL_NAME = "BreastCancerClassifier"
MODEL_ALIAS = os.getenv("MODEL_ALIAS", "champion")
print(f"Using MODEL_ALIAS={MODEL_ALIAS}")

model_uri = f"models:{MODEL_NAME}@{MODEL_ALIAS}"
print(f>Loading model from: {model_uri}")

# Load model referenced by alias
model = mlflow.pyfunc.load_model(model_uri)
print(model)

class PredictionInput(BaseModel):
    features: list[float]

@app.post("/predict")
def predict(input_data: PredictionInput):
    if len(input_data.features) != 30:
        return {"error": "Expected 30 features"}
    arr = np.array(input_data.features).reshape(1, -1)
    prediction = model.predict(arr)
    return {
        "prediction": int(prediction[0])
    }

```

This is production-style design. Now run uvicorn to expose this inference:

```
uvicorn app:app --reload --port 8000
```

Now test:

```

curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [1,2,3,4,5,6,7,8,9,1,2,3,4,5,6,7,8,9,1,2,3,4,5,6,7,8,9,1,2,3]}'

```

You now have a local inference service.